Pràctica 02 - Desplegament amb Docker build

Objectius	Desplegar una aplicació amb Docker
Tipus de Pràctica	Activitat pràctica Individual
Material:	Lliure
Temps	30 minuts de treball a classe com a mínim
Lliurament	S'ha de lliurar una còpia d'aquest document amb les respostes a la tasca corresponent del Google ClassRoom No s'acceptaran lliuraments per mail o fora de plas Documenta el procés amb captures de pantalla i explica amb les teves paraules què has fet i quins problemes has trobat. Les captures han de contenir els elements necessaris per identificar l'autoria i veracitat No es permet alterar posteriorment les captures de pantalla No es permet cap copia literal de cap font, ni en codi font ni en explicacions. El treball pot ser avaluat oralment.

Introducció

En aquesta pràctica farem ús d'instruccions pas a pas sobre com començar a utilitzar Docker en aquest aspectes principals:

Construir i executar una imatge com a contenidor.
Implementar aplicacions Docker utilitzant diversos contenidors amb una base de dades.

Exercici 1 - Instal·lar el codi font de l'aplicació

Treballarem amb un senzill gestor de llistes de tasques pendents que s'executa amb Node.js. Si no esteu familiaritzats amb Node.js, no us preocupeu. Aquesta pràctica no requereix cap experiència prèvia amb JavaScript.

Requisits previs:

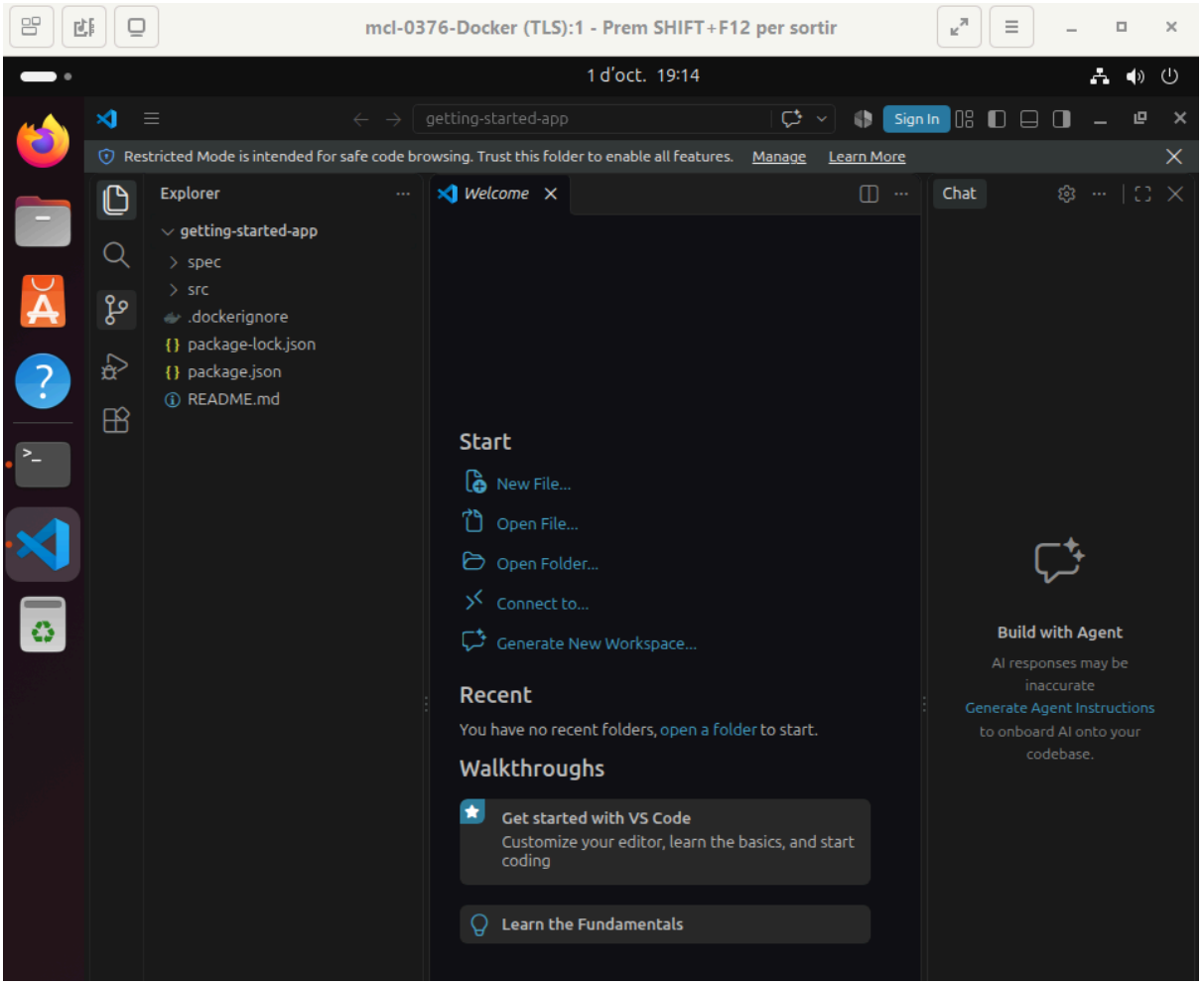
- Has d'haver instal·lat el laboratori al ISARD.
- Has d'instal·lar el client Git amb apt install.
- Un editor de text per editar fitxers. Opcionalment podeu utilitzar Visual Studio Code.

Baixa l'aplicació
Abans de poder executar l'aplicació, cal que instal·leu el codi font de l'aplicació al laboratori.

1. Clona el repositori getting-started-app amb l'ordre següent:

Instal·lo el client git i l'editor Visual Studio Code per clonar el repositori oficial de l'aplicació des de GitHub i obrir la carpeta del projecte, d'aquesta manera disposo del codi font de Node.js a la meva màquina local per poder consultar-lo i modificar-lo

```
mcl@ubuntu-24:~/getting-started-app$ history
    1  clear
    2  sudo apt update
    3  sudo apt install git -y
    4  sudo snap install code --classic
    5  git clone https://github.com/docker/getting-started-app.git
    6  cd getting-started-app/
    7  code .
    8  history
```



2. Visualitza el contingut del repositori clonat.

Executo l'ordre ls -la dins del directori getting-started-app per mostrar tots els fitxers i directoris i així comprovar que el repositori s'ha descarregat correctament amb tots els fitxers necessaris

```
mcl@ubuntu-24:~/getting-started-app$ ls -la
total 152
drwxrwxr-x  5 mcl mcl   4096 d'oct.    1 19:12 .
drwxr-x--- 18 mcl mcl   4096 d'oct.    1 19:13 ..
-rw-rw-r--  1 mcl mcl     24 d'oct.    1 19:12 .dockerignore
drwxrwxr-x  8 mcl mcl   4096 d'oct.    1 19:12 .git
-rw-rw-r--  1 mcl mcl    541 d'oct.    1 19:12 package.json
-rw-rw-r--  1 mcl mcl 118912 d'oct.    1 19:12 package-lock.json
-rw-rw-r--  1 mcl mcl    269 d'oct.    1 19:12 README.md
drwxrwxr-x  4 mcl mcl   4096 d'oct.    1 19:12 spec
drwxrwxr-x  5 mcl mcl   4096 d'oct.    1 19:12 src
mcl@ubuntu-24:~/getting-started-app$
```

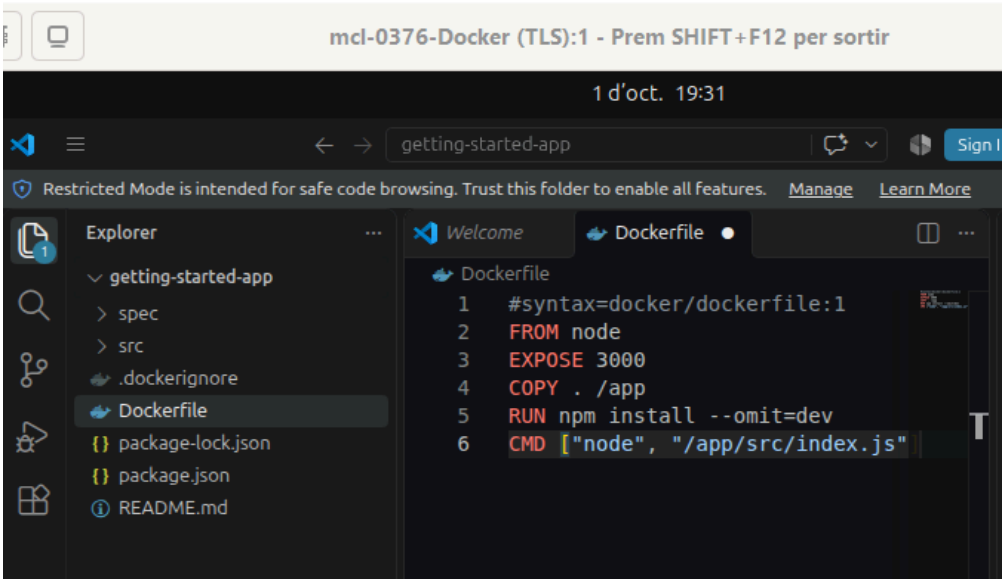
Documenta el procés amb captures de pantalla i explica amb les teves paraules què has fet i quins problemes has trobat.

Exercici 2. Crea la imatge de l'aplicació

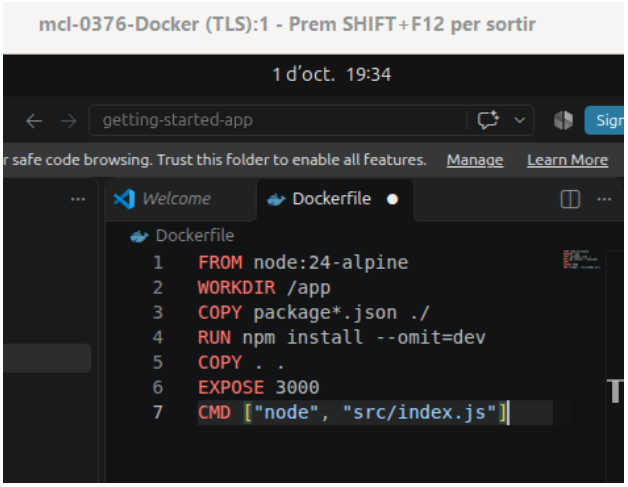
Per crear la imatge, haureu d'utilitzar un Dockerfile. Un Dockerfile és simplement un fitxer basat en text sense extensió de fitxer que conté un script d'instruccions. Docker utilitza aquest script per crear una imatge de contenidor.

1. Al directori de l'aplicació, a la mateixa ubicació que el fitxer package.json, crea un fitxer anomenat Dockerfile amb el contingut següent:

Genero el Dockerfile dins del visual studio, i li enganxo el contingut



2. Modifica el Dockerfile i aplica els principis de bones pràctiques que has vist a clase, justifica si aplica o no cada punt del material vist a clase.



Modifico el Dockerfile inicial canviant la imatge base a “node:24-alpine” per reduir la mida de la imatge i millorar la seguretat. Afegeixo “WORKDIR /app” per tenir un directori de treball definit i reordeno les ordres copiant primer el “package*.json” abans de la resta del codi. Això ho faig per aprofitar la memòria cau de Docker i evitar reinstal·lar dependències si només canvio el codi font

Documenta el procés amb captures de pantalla i explica amb les teves paraules què has fet i quins problemes has trobat.

Exercici 3. Construeix la imatge

- Etiqueta la imatge generada amb el tag “getting-started”

Al terminal, assegura't que ets al directori correcte.

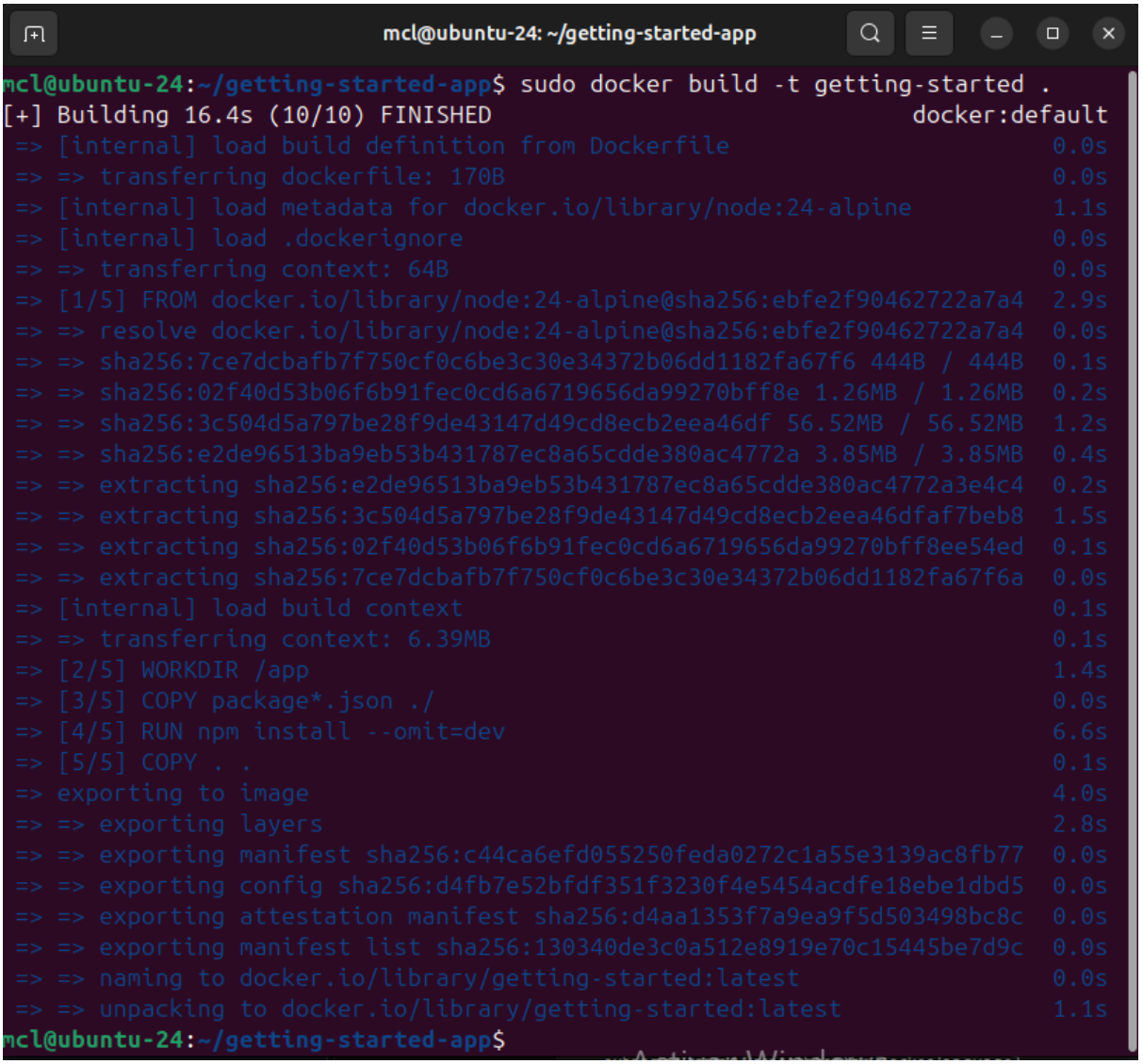
El docker build utilitza el fitxer Dockerfile per crear una nova imatge. Potser heu notat que Docker ha descarregat moltes "capes". Això és degut a que heu indicat al constructor que volies començar des de la imatge base node:24-alpine. Però, com que no la tens a la teva màquina, Docker havia de descarregar la imatge.

Després que Docker hagi descarregat la imatge, les instruccions del fitxer Dockerfile s'han copiat a l'aplicació i s'han utilitzat npm per instal·lar les dependències de l'aplicació.

Finalment, has d’etiquetar la teva imatge. Pensa en això com un nom llegible per a la imatge final. Com que has anomenat la imatge getting-started, pots fer referència a aquesta imatge quan executis un contenidor.

El . al final de l' docker build indica a Docker que ha de buscar el Dockerfile al directori actual.

Executo l'ordre “sudo docker build -t getting-started .” des del directori del projecte per construir la imatge de contenidor utilitzant les instruccions del “Dockerfile”. L'opció “-t getting-started” assigna un nom a la imatge i el punt . indica que el context de construcció és el directori actual, sobretot, abans d'executar la comanda, el Dockerfile a de ser guardat, a visual studio dins del archiu amb ctrl+s.



Documenta el procés amb captures de pantalla i explica amb les teves paraules què has fet i quins problemes has trobat.

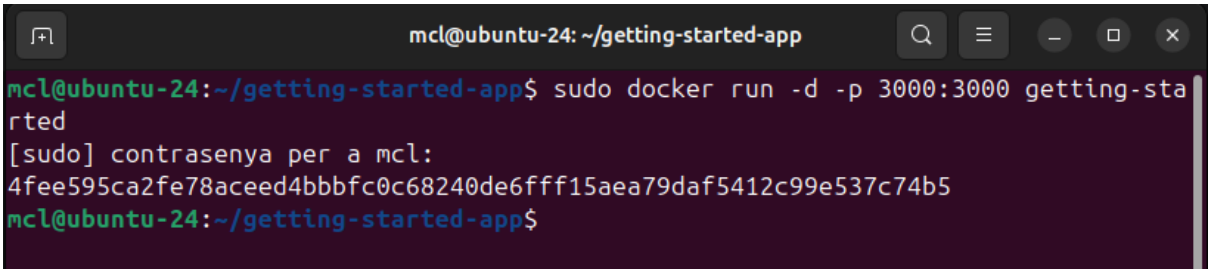
Exercici 4. Inicia l'aplicació

1. Executeu el contenidor amb el docker run i especifiqueu el nom de la imatge que acabeu de crear:

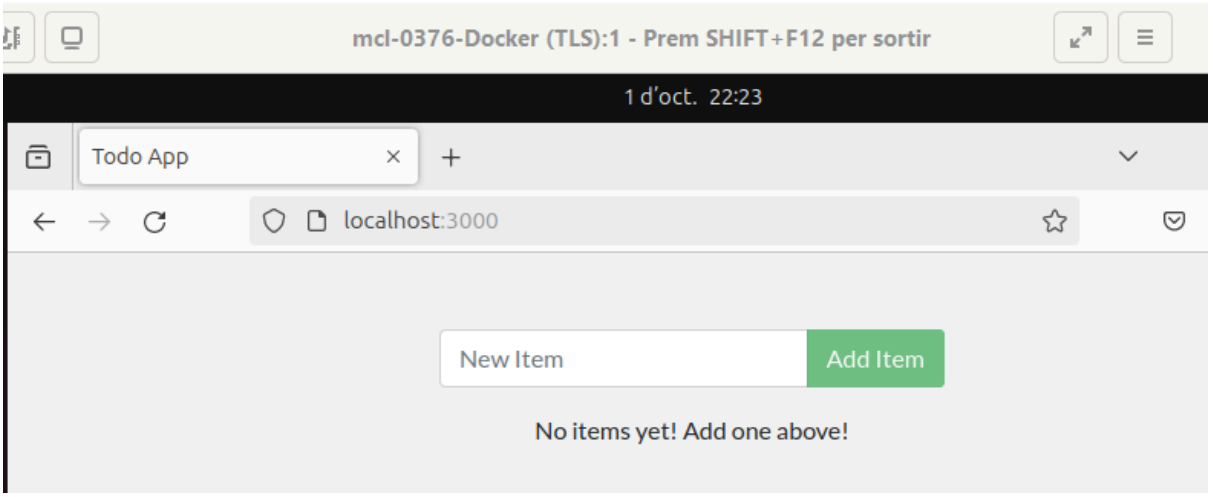
a. Executa el contenidor en segon pla. Això significa que Docker inicia el contenidor i et retorna a l'indicador del terminal. A més, no mostra els registres al terminal.

b. Crea una correspondència de ports entre l'amfitrió i el contenidor per tal que el port 3000 del contenidor redireccioni al 3000(localhost:3000) de l'amfitrió. Sense la correspondència de ports, no podries accedir a l'aplicació des de l'amfitrió.

A la captura s'observa com executo la comanda "sudo docker run -d -p 3000:3000 getting-started" per iniciar el contenidor. Docker crea el contenidor, enllaça el port 3000 del sistema amfitrió amb el del contenidor i retorna l'ID únic del procés generat. En utilitzar l'opció -d, el contenidor passa a executar-se en segon pla i la terminal torna a estar disponible immediatament.



2. Després d'uns segons, obre el navegador web a http://localhost:3000 . Hauries de veure l'aplicació amb la llista de tasques buida.



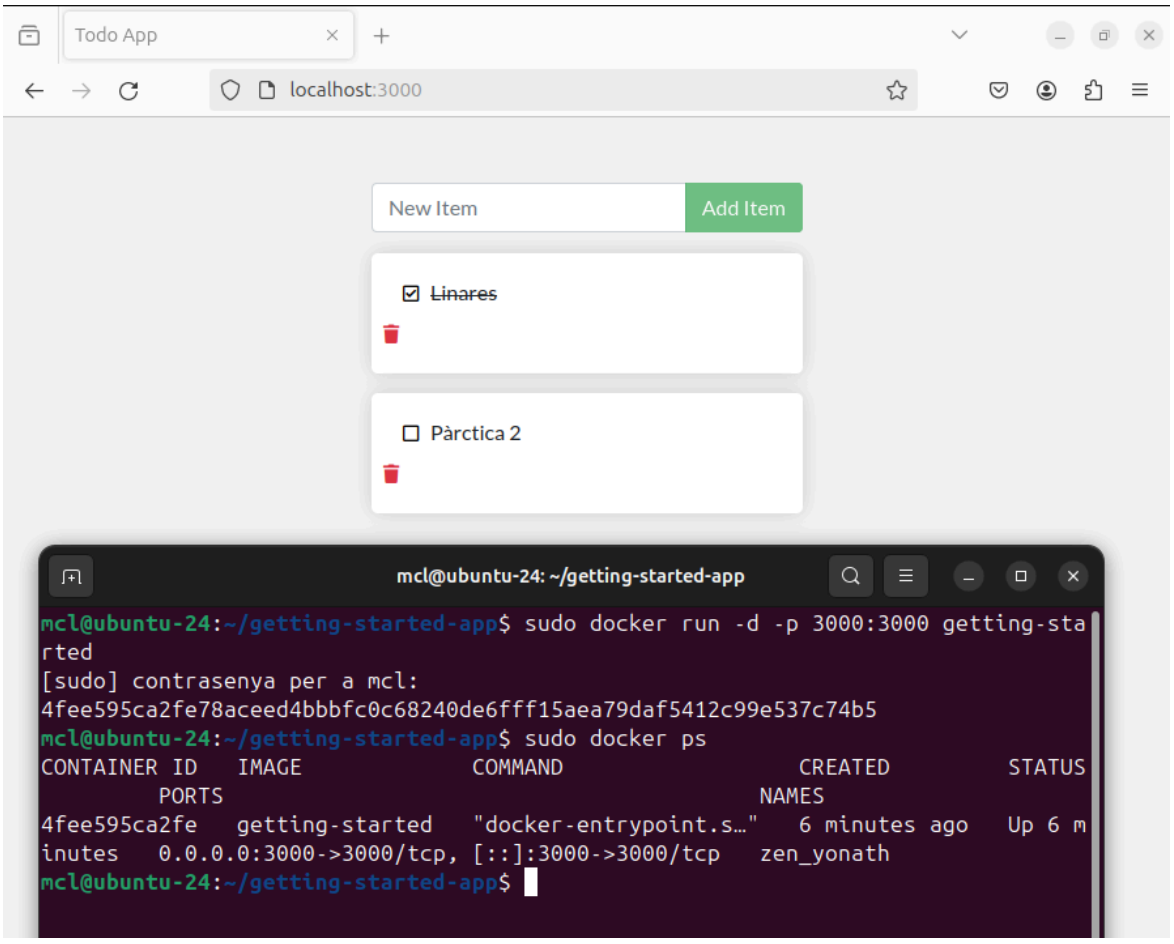
3. Afegeix un element o dos i comprova que funciona com esperes. Pots marcar els elements com a complets i eliminar-los. El frontend està emmagatzemant correctament els elements al backend.

Afegeixo Linares i Pràctica 2, com a elements i per comprovar que el frontend esta fent la seva feina, actualitzo al web i els cambis del estat dels elements segueixen alla

En aquest punt, tens un gestor de llistes de tasques pendents en execució amb alguns elements.

4. Fes una ullada ràpida als contenidors, hauries de veure com a mínim un contenidor en execució que utilitza la imatge getting-started i el port 3000.

L'execució de la comanda "sudo docker ps" per llistar els contenidors meus actius al sistema. A la sortida es confirma que hi ha un contenidor en execució basat en la imatge "getting-started". Es verifica que esta en marxa des de fa 6 minuts i que té el mapeig de ports "0.0.0.0:3000->3000/tcp" correctament assignat, la qual cosa permet l'accés a l'aplicació des del navegador



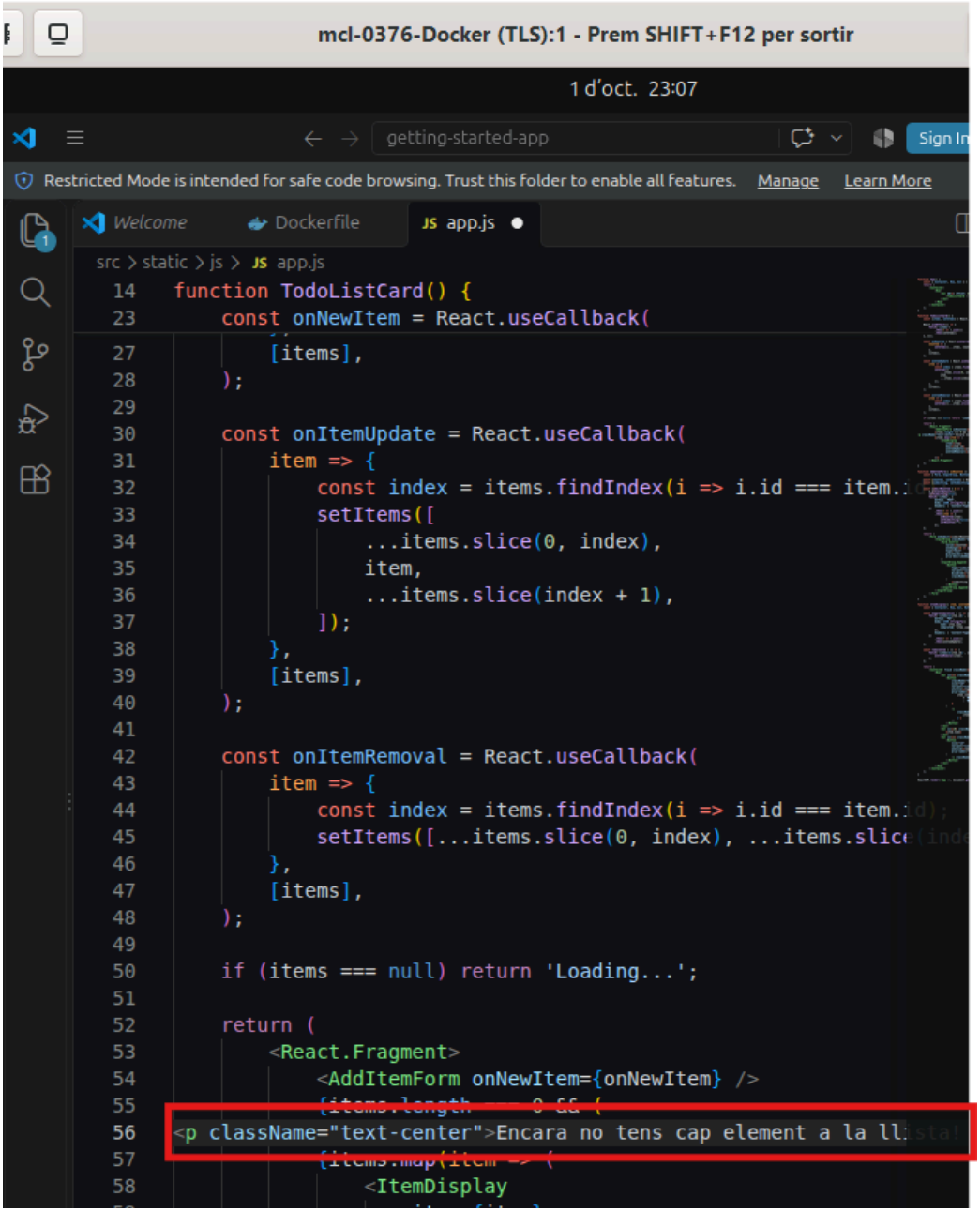
Documenta el procés amb captures de pantalla i explica amb les teves paraules què has fet i quins problemes has trobat.

Exercici 5. Actualitza l'aplicació

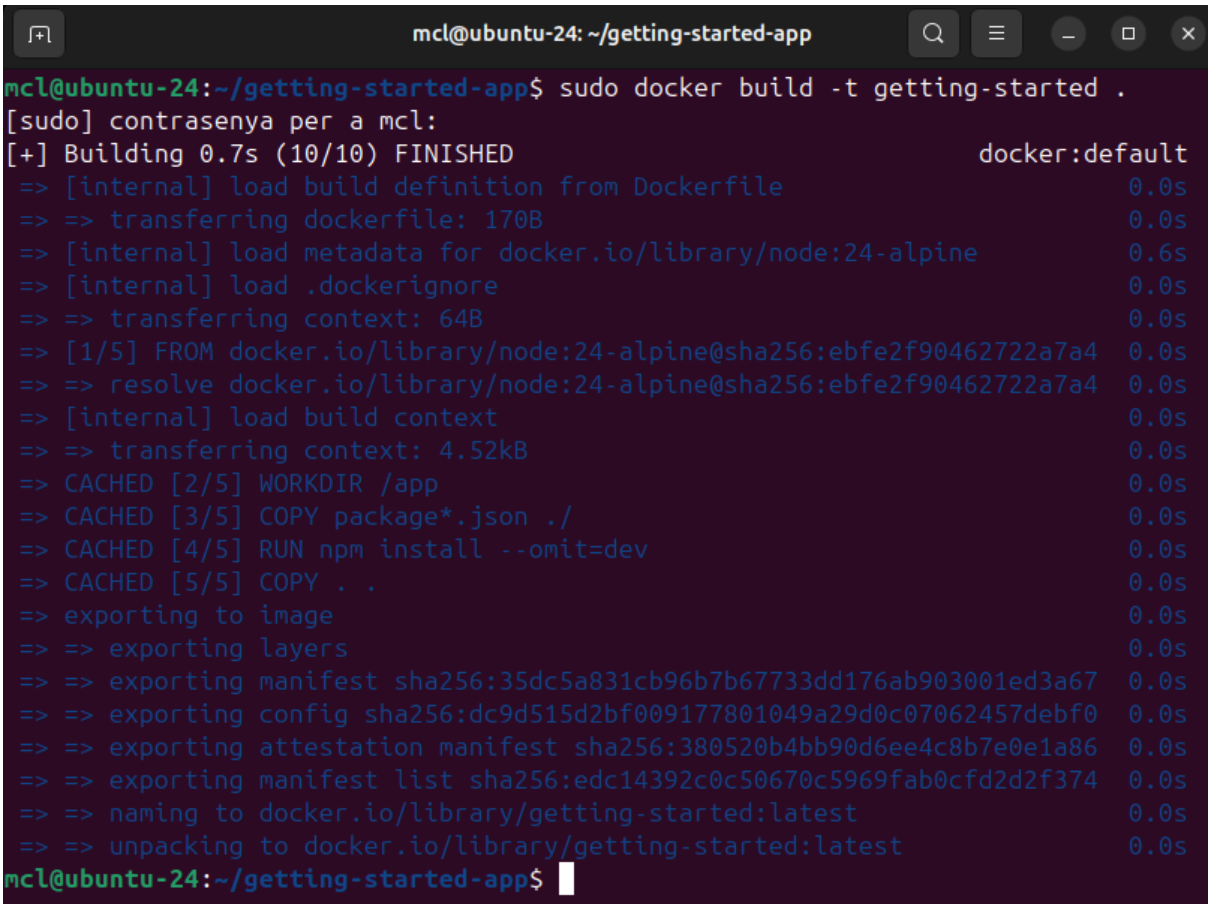
Canvia el "text buit" quan no tingueu cap element de la llista de tasques pendents a "Encara no teniu cap element de tasca pendent! Afegeix-ne un a dalt!".

1. Al src/static/js/app.js fitxer, actualitzeu la línia 56 per utilitzar el nou text buit.

- No items yet! Add one above!
+ Encara no tens cap element a la llista! Afegeix-ne un!

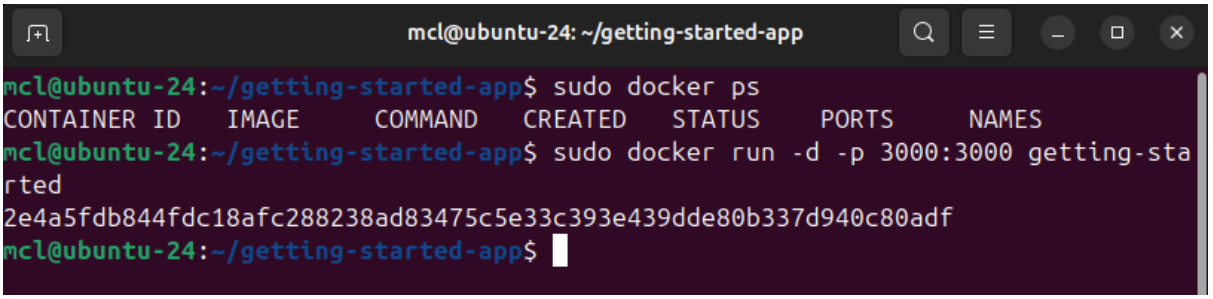


2. Crea la teva versió actualitzada de la imatge com en l'exercici 3.



3. Inicia un contenidor nou utilitzant el codi actualitzat com en l'exercici 4.

No em dona cap error, ja que el contenidor antic ja estava aturat d'abans perquè he hagut d'aturar la màquina virtual



- Probablement has vist un error com aquest:

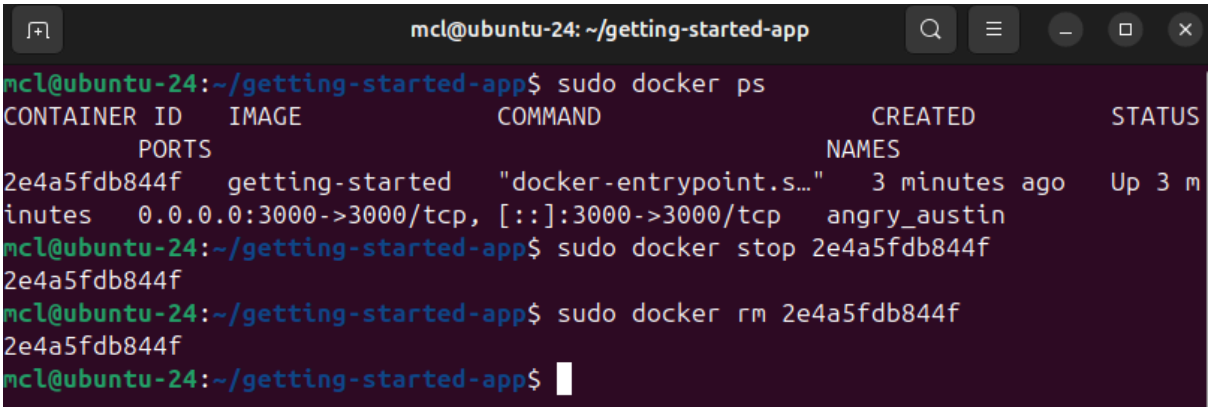
docker: Error response from daemon: driver failed programming external connectivity on endpoint laughing_burnell (bb242b2ca4d67eba76e79474fb36bb5125708ebdabd7f45c8eaf16caaabde9dd): Bind for 127.0.0.1:3000 failed: port is already allocated.

L'error s'ha produït perquè no podeu iniciar el contenidor nou mentre el contenidor antic encara està en funcionament. El motiu és que el contenidor antic ja utilitza el port 3000 de l'amfitrió i només un procés de la màquina (inclosos els contenidors) pot escoltar un port específic. Per solucionar-ho, heu de suprimir el contenidor antic.

4. Elimina el contenidor antic

Per eliminar un contenidor, primer s'ha d'aturar. Un cop s'hagi aturat, el podras eliminar.

a. Obté l'ID del contenidor mitjançant docker ps.
b. Fes servir docker stop per aturar el contenidor.
c. Un cop el contenidor s'hagi aturat, el pots eliminar mitjançant docker rm.



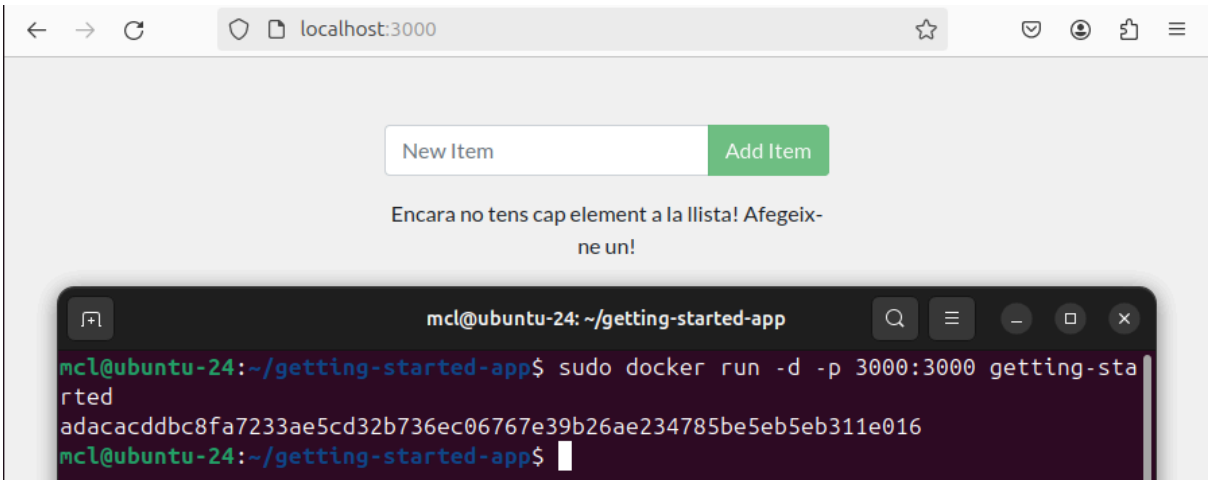
Nota:
Pots aturar i eliminar un contenidor en una sola ordre afegint el senyalador force al docker rm

Per fer-ho tot d'una sola comanda seria aixins: sudo docker rm -f "Id contenidor"

5. Inicia el contenidor d'aplicacions actualitzat

Ara, inicia l'aplicació actualitzada amb docker run.
Actualitza el navegador a http://localhost:3000 i hauries de veure el text d'ajuda actualitzat.

Com veiem, apareix el missatge actualitzat, ja no surt en anglès, sinó en català



Documenta el procés amb captures de pantalla i explica amb les teves paraules què has fet i quins problemes has trobat.